MOUNTAIN V2V PROJECT

Date: 1/9/23

iTETRIS V2V Project — Progress Report & Repro Guide

Author: Mallikarjun, Nithin, Bhargav Reddy, Mayank
Project: Accident collision prediction and avoidance using V2V/V2X in mountain curve roads (iTETRIS integration)
Date: 12/10/25

---

1. Summary — current status

This document records **every step you performed** while setting up the simulation + ML pipeline so a colleague (or you in the future) can reproduce the environment and the experiments exactly. It includes all commands executed, files created/modified, problems encountered and how they were solved, and the current state of each component.

High level completed items:

- SUMO map (mountain hairpin) prepared: `simulation/map/mountain.net.xml`, `mountain.osm` retained as source.
- SUMO route & trips generation (routes.rou.xml, trips.trips.xml) — randomTrips used; final `mountain.rou.xml` produced.
- SUMO+TraCI logging script created: `simulation/scripts/log_vehicle_data.py` — able to run SUMO simulation and log vehicle states.
- Python scripts for trip generation: `simulation/scripts/generate_trips.py` — adjusted path handling and fixed `additional.xml` handling.
- ML pipeline: feature and label CSVs placed under `ml_model/dataset/`, LSTM training scripts written and iterated: `ml_model/notebooks/train_model.py` (multiple upgraded versions) and `ml_model/models/lstm_v2v.h5` saved.
- Performance reporting script: `ml_model/notebooks/perf_report.py` updated to use pandas and robust relative paths. Comprised classification report, confusion matrix, ROC.
- Class imbalance handling added to training (class weights); history saved to `ml_model/models/history.pkl`.
- NS-3 + NetAnim installed inside WSL2 (Ubuntu) using `ns-allinone-3.39`; NetAnim built; NS-3 verified (`hello-simulator`).

- Work-in-progress: Python bindings for ns-3 were not initially present; debugging steps and PYTHONPATH set to `.../ns-3.39/build/lib` for reproducibility.

---

2. Project layout (current)

```
itetris-v2x-project/
├── README.md
├── environment_setup/
│   ├── install_sumo_ns3.sh
│   └── install_itetris.md
├── simulation/
│   ├── map/
│   │   ├── mountain.osm
│   │   ├── mountain.net.xml
│   │   ├── routes.rou.xml
│   │   ├── trips.trips.xml
│   │   └── simulation.sumocfg
│   ├── output/
│   │   ├── vehicle_data.csv
│   │   └── logs/
│   └── scripts/
│       ├── generate_trips.py
│       ├── log_vehicle_data.py
│       └── analyze_collisions.py
├── ml_model/
│   ├── dataset/
│   │   ├── features.csv
│   │   └── labels.csv
│   ├── notebooks/
│   │   ├── feature_engineering.ipynb
│   │   ├── train_model.py
│   │   └── evaluation.ipynb
│   ├── models/
│   │   ├── baseline_ttc.py
│   │   ├── lstm_v2v.h5
│   │   └── scaler.pkl
│   └── serving/
│       └── serve_model.py
└── docs/
    ├── design_doc.md
    └── presentation.pptx
```

---

3. Detailed chronological log & commands executed

Below are the exact commands run (or attempted) and the important outputs / errors, in chronological order. Use these to reproduce the same environment from scratch.

3.1 SUMO: map, netconvert, randomTrips and troubleshooting

Environment: Windows host (PowerShell), SUMO `sumo-1.24.0` installed under `C:\sumo-1.24.0`.

Problems encountered & fixes:

- `netconvert --osm-files mountain.osm -o mountain.net.xml` failed initially with `Could not open osm-file 'mountain.osm'` — path issue. Resolved by running `netconvert` from the directory containing `mountain.osm`.
- SUMO environment variable warnings: `SUMO_HOME` not set. Temporary fix: run direct binary `C:\sumo-1.24.0\bin\netconvert.exe` or export `SUMO_HOME` in PowerShell: `setx SUMO_HOME "C:\sumo-1.24.0"` (restart PowerShell).
- `netcheck` not found in PowerShell: use full path `C:\sumo-1.24.0\bin\netcheck.exe` or add SUMO `bin` to PATH.

randomTrips.py usage:

- To generate routes/trips (Python script requires `sumolib`/`traci` Python packages or correct `PYTHONPATH` pointing to SUMO tools):

Windows examples:

```
python "C:\sumo-1.24.0\tools\randomTrips.py" -n mountain.net.xml -r
mountain.rou.xml -e 100
```

- Error: `ModuleNotFoundError: No module named 'sumolib'`. Fix: ensure SUMO tools installed into Python path. Option: run randomTrips via SUMO's Python environment or copy `sumolib` and `traci` into the tools folder of the Python environment. Alternatively add SUMO tools path to `PYTHONPATH` in PowerShell before running: `set PYTHONPATH=%SUMO_HOME%\tools`.

Generated files: `mountain.net.xml`, `mountain.rou.xml` (100 trips), `trips.trips.xml`.

3.2 generate_trips.py & additional.xml issues

Symptom: Running `python simulation/scripts/generate_trips.py` produced `FileNotFoundError` complaining about `map/additional.xml` path.

Fix applied: Updated script path handling to use project-root-relative paths; created `map/additional.xml` as required. The corrected code uses `os.path.abspath(os.path.join(...))` and `PROJECT_DIR` to ensure stable paths irrespective of current working directory.

3.3 log_vehicle_data.py & TraCI

Symptom: `Could not access configuration 'map/simulation.sumocfg'` and subsequent TraCI connection issues.

Fixes:

- Ensure `simulation.sumocfg` path is correct and referenced in the script via project-root-relative `PROJECT_DIR` variable.

- Ensure SUMO server (`sumo` or `sumo-gui`) is launched with the same configuration and TraCI port as the script (example: `sumo-gui -c map/simulation.sumocfg --remote-port 6023`).
- If sudden `Connection closed by SUMO` errors occur, check that only one TraCI instance uses the port, add retries and proper `traci.start([...])` error handling.

3.4 ML environment setup (TensorFlow) and GPU troubleshooting

Local machine: Windows laptop (Ryzen 5600H, GTX 1650, AMD integrated GPU present)

Steps attempted and commands used:

- Created venv: `python -m venv C:\Users\mayan\Desktop\V2V\v2v_gpu_env` and later a CPU venv `v2v_cpu_env`.
- Tried `pip install tensorflow-cpu==2.16.1` but pip could not find matching wheel for Python/OS combination.
- Installed `tensorflow==2.20.0` (worked) but TF showed `GPUs detected: []`.

Conda attempt:

- Installed Miniconda/Anaconda, accepted channel ToS (`conda tos accept ...`), attempted `conda create -n v2v_tf python=3.11` and `conda install tensorflow=2.20 -c conda-forge` — ran into package availability issues on conda-forge for Windows. Final decision: use a `venv` and CPU TensorFlow to avoid Windows GPU packaging pain.

Final working env (CPU):

```
python -m venv v2v_cpu_env
.\v2v_cpu_env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install tensorflow scikit-learn pandas numpy joblib matplotlib
```

Mixed precision / GPU notes:

- Tried enabling mixed precision; on CPU it is harmless but not accelerating.
- Attempted to use NVIDIA GPU but TF kept detecting AMD driver. On Windows, GPU support requires matching CUDA/cuDNN + TF build and is error-prone; using CPU for now.

3.5 Training script development and improvements

Files created/updated: `ml_model/notebooks/train_model.py` (several versions). Key features implemented:

- Loading `features.csv` and `labels.csv` using pandas
- StandardScaler saved (`scaler.pkl`) using joblib
- LSTM model (64 units) with dropout, compiled with `binary_crossentropy` and `accuracy`
- Optional mixed precision handling (no-op on CPU)
- EarlyStopping callback with `patience=5`
- Saving model to `ml_model/models/lstm_v2v.h5` and history to `history.pkl`

- Class imbalance handling via `sklearn.utils.class_weight.compute_class_weight` and passing `class_weight` to `model.fit`

Commands to run training:

```
# from project root
python ml_model/notebooks/train_model.py
```

Observed result: Model trained, early-stopped at 39 epochs in one run; test accuracy ~0.942 but minority class recall extremely low.

3.6 Performance evaluation & class imbalance

Perf script: `ml_model/notebooks/perf_report.py` updated to use pandas for robust CSV parsing and project-root-relative paths. It prints classification report, confusion matrix, ROC curve and optionally plots training history if `history.pkl` exists.

Key evaluation result: The model achieved high accuracy (0.94) but extremely poor recall on minority class. To address this you added class weights and suggested oversampling (SMOTE), threshold tuning, and alternative metrics (PR-AUC) in the code.

3.7 NS-3 & NetAnim installation (WSL2 Ubuntu)

WSL2 status: Ubuntu installed & updated. Steps executed below and outputs collected are recorded here — run in WSL terminal.

Commands executed (summary):

```
# install deps
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential gcc g++ python3 python3-dev python3-pip git
cmake mercurial \
    libsqlite3-dev libxml2-dev libgtk-3-dev libharfbuzz-dev libeigen3-dev python3-
pybindgen qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools bzip2 pkg-config

# download ns-3 allinone
mkdir -p ~/ns3 && cd ~/ns3
wget https://www.nsnam.org/releases/ns-allinone-3.39.tar.bz2
tar -xjf ns-allinone-3.39.tar.bz2
cd ns-allinone-3.39
./build.py --enable-examples --enable-tests
# netanim built as part of the process
# test ns-3
cd ns-3.39
./ns3 run hello-simulator

# netanim
cd ../netanim-3.109
./NetAnim
```

Troubleshooting notes:

- Needed `bzip2` to extract archive
- CMake initial error complaining about no CXX compiler: resolved by installing `g++/build-essential`.
- CMake RPATH errors with Ninja: worked around by building inside source tree (default `build.py` behavior) or passing cmake flags.
- Python ns-3 bindings were not found at first — resolved by ensuring `python3-pybindgen` and `python3-dev` were installed, reconfiguring and exporting `PYTHONPATH` to `~/ns-allinone-3.39/ns-3.39/build/lib`.

Successful checks done:

- `./ns3 run hello-simulator` printed `Hello Simulator`.
- NetAnim compiled and runs.

---

4. Reproducible command list

Below is a minimal sequence you (or your guide) can run on a fresh Windows 10/11 machine to reproduce the same setup. Copy-paste in order; WSL2/Ubuntu approach is used for NS-3.

4.1 Windows / PowerShell (install SUMO, prepare project)

```
# Set SUMO_HOME (adjust path to your SUMO installation)
setx SUMO_HOME "C:\sumo-1.24.0"
# Optionally add to PATH
setx PATH "%PATH%;C:\sumo-1.24.0\bin"
# From PowerShell, run SUMO GUI to verify
sumo-gui --version
```

4.2 Python venv for ML (Windows PowerShell)

```
cd C:\Users\mayan\Desktop\V2V\Mountain_v2v_project
python -m venv v2v_cpu_env
.\v2v_cpu_env\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install tensorflow scikit-learn pandas numpy joblib matplotlib seaborn
# run training (CPU)
python ml_model/notebooks/train_model.py
```

4.3 WSL2 / Ubuntu (NS-3 & NetAnim)

Open an **An Ubuntu (WSL)** terminal and run:

```
# update
sudo apt update && sudo apt upgrade -y
# install general deps
```

```
sudo apt install -y build-essential gcc g++ python3 python3-dev python3-pip git
cmake mercurial bzip2 pkg-config
# optional libs
sudo apt install -y libsqlite3-dev libxml2-dev libgtk-3-dev libharfbuzz-dev
libeigen3-dev python3-pybindgen qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools

# download & extract ns-allinone

mkdir -p ~/ns3 && cd ~/ns3
wget https://www.nsnam.org/releases/ns-allinone-3.39.tar.bz2
tar -xjf ns-allinone-3.39.tar.bz2
cd ns-allinone-3.39
# build ns-3 + netanim
./build.py --enable-examples --enable-tests
# test
cd ns-3.39
./ns3 run hello-simulator
# set python path (adjust python version number directory)
export PYTHONPATH=$PWD/build/lib:$PYTHONPATH
python3 -c "import ns.core; import ns.network; print('ns3 python bindings OK')"
# launch NetAnim GUI (optional)
cd ../netanim-3.109
./NetAnim
```

4.4 Re-run SUMO example with TraCI logging (from Windows or WSL; adjust paths)

- Ensure `simulation/map/simulation.sumocfg` points to `mountain.net.xml` and `mountain.rou.xml`.
- Launch SUMO in server mode (TraCI):

  ```
  sumo-gui -c /path/to/simulation.sumocfg --remote-port 6023
  ```

- From Python script, connect via TraCI to port 6023 and log vehicle states.

---

5. Scripts & file descriptions (what each script does)

- `simulation/scripts/generate_trips.py`: Generates trips/routes using SUMO `randomTrips.py` logic and writes `routes.rou.xml`. Fixed to use project-root absolute paths and added an `additional.xml` template that sets vehicle types and lane attributes (need to include file content in repo).
- `simulation/scripts/log_vehicle_data.py`: Launches SUMO via TraCI and logs timestamped vehicle states (id, x,y,lat,lon,edge, lane, speed, acceleration, heading). Writes `simulation/output/vehicle_data.csv`.
- `ml_model/notebooks/train_model.py`: Loads features & labels, scales with `StandardScaler` (saved), builds LSTM, fits with class weights, saves model + history. Supports mixed precision if GPU found.
- `ml_model/notebooks/perf_report.py`: Loads scaler/model/dataset and computes classification report, confusion matrix, ROC plot, and training curves if history exists.
- `ml_model/models/baseline_ttc.py`: Example baseline Time-To-Collision rule-based predictor (keeps in repo as baseline comparator).

6. Known issues & lessons learned

1. **SUMO `sumolib` Python imports:** When running SUMO tools via Python, ensure SUMO tools are on `PYTHONPATH` or use the bundled Python interpreter.
2. **TF GPU on Windows is hard:** Matching CUDA/cuDNN/TensorFlow versions caused long delays; using CPU TF in a venv is acceptable for development and debugging.
3. **NS-3 build on WSL requires many deps:** `bzip2`, `g++`, `pybindgen`, GTK, libxml, libsqlite3, Qt dev packages — missing any leads to errors.
4. **Path handling in scripts:** Always compute `PROJECT_DIR` using `os.path.dirname(__file__)` up two levels; do not rely on current working directory.

---

7. Next steps & recommended priorities

1. **Finish NS-3 Python bindings** and validate `ns` imports in Python inside WSL (follow repro commands in section 4.3).
2. **Install iTETRIS** and use its example for SUMO–NS-3 integration.
3. **Create a minimal integrated demo**: two vehicles on a hairpin curve broadcasting speed/acc via DSRC; log packet send/recv events.
4. **Visualize packet transfer**: create NetAnim trace from NS-3 or parse logs to animate in matplotlib/plotly.
5. **Generate labeled collision dataset** by configuring challenging speeds on curves and simulating network delays (to produce positive collision labels) so the ML model has non-zero collision examples.
6. **Improve ML pipeline**: sequence/timesteps >1, more complex LSTM or transformer, resampling/SMOTE and threshold tuning, precision-recall optimization.

---

8. Attachments & snippets to include in repo

Include these small files (if not already present) in `environment_setup/`:

- `install_ns3_wsl.sh` — shell script automating NS-3 + NetAnim dependencies and build (copy the commands in 4.3).
- `install_sumo_on_windows.md` — short notes for setting SUMO and SUMO_HOME on Windows.
- `run_itetris_demo.sh` — wrapper to launch SUMO (TraCI port), then NS-3/iTETRIS scenario.

---

9. How to reproduce entire work from scratch (summary)

1. Prepare Windows host: install SUMO and set `SUMO_HOME`.
2. Create Python venv for ML (`v2v_cpu_env`), `pip install` required packages.

3. Prepare SUMO map + routes; generate trips using `randomTrips.py`.
4. Build NS-3 inside WSL (follow 4.3 commands).
5. Install iTETRIS and run example hairpin scenario.
6. Run the logging script to collect `vehicle_data.csv` and produce `features.csv` and `labels.csv`.
7. Run `train_model.py` to train LSTM and `perf_report.py` to evaluate.

---

10. Contact & next actions

If you want, I will:

- Add `install_ns3_wsl.sh` and `run_itetris_demo.sh` scripts to the repo.
- Produce a minimal `iTETRIS + NS-3 + SUMO` example scenario code (Python) for two vehicles swapping speed/accel on a hairpin curve and generating NetAnim traces.